강의 · 딥러닝

신경망 기초

뉴런 하나에서 학습된 신경망까지

직관을 먼저, 그림은 내내, 수식은 꼭 필요한 곳에만

뉴런과 활성화 함수 · 층과 표현력 · 손실과 경사하강법
역전파 · 과적합과 실무 기본값

딥러닝의 거의 전부를 세 가지 생각으로 설명할 수 있다

이후 과정의 모든 내용은 이 세 가지의 변형이다

1

뉴런은 가중 투표 뒤에 꺾임을 더한 것이다

선형 점수에 비선형 활성화 함수를 붙인 것이 유일한 구성 요소다.

2

꺾임을 쌓으면 신경망은 어떤 모양이든 그릴 수 있다

각 층이 공간을 다시 빚어 다음 층의 일을 더 쉽게 만든다.

3

학습은 손실 곡면을 따라 내려가는 것이다

역전파는 기울기를 계산하고, 경사하강법은 한 걸음 내딛는다.

모형 = 구조(함수의 모양) · 학습 = 탐색(어떤 함수인가)

네 가지 측정값으로 붓꽃 품종을 맞힌다

붓꽃 데이터셋(Fisher 1936): setosa, versicolor, virginica





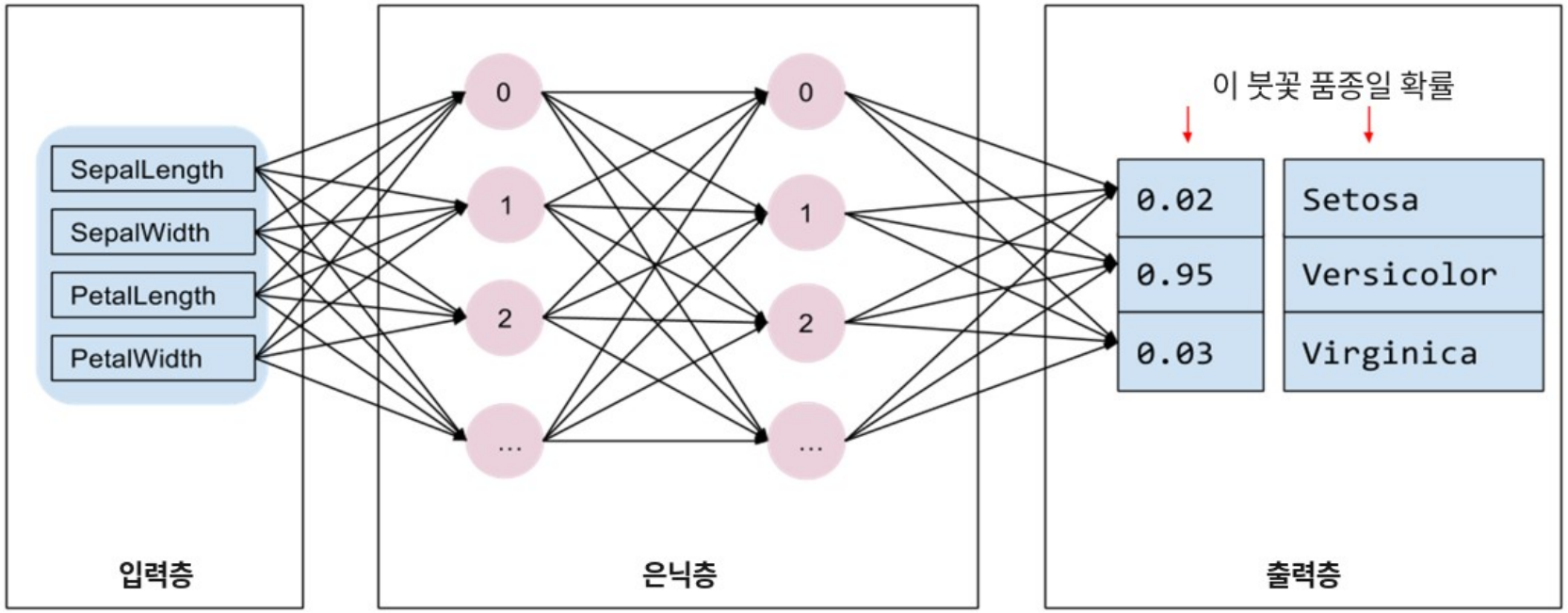


과제

- 입력: 꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비(cm)
- 출력: 세 품종 중 하나; 꽃 150송이, 품종마다 50송이

측정값 넷이 들어가고 확률 셋이 나온다

ReLU 유닛 10개짜리 은닉층 2개; softmax가 출력 3개의 합을 1로 만든다



Keras 코드 · 완전 연결 = Dense 층

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation="relu", input_shape=(4,)),   # 입력 4개
    tf.keras.layers.Dense(10, activation="relu"),
    tf.keras.layers.Dense(3)])                                        # 클래스 점수 3개 -> softmax
```

01

1부 · 구조

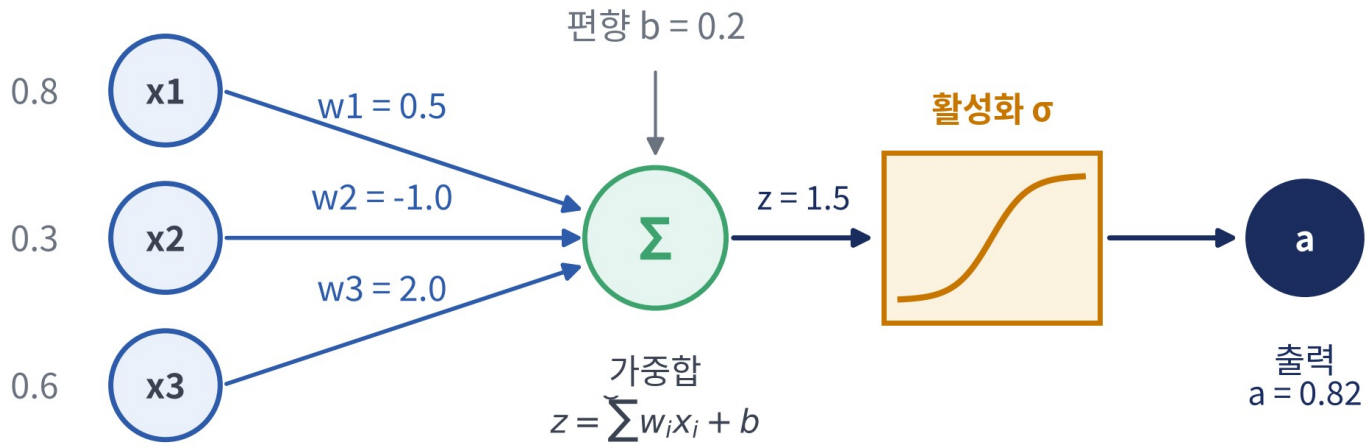
신경망의 구성 요소

뉴런, 활성화 함수, 층, 그리고 깊이가 중요한 이유

뉴런은 증거에 가중치를 매긴 뒤 얼마나 발화할지 정한다

계산 예: 입력 (0.8, 0.3, 0.6), 가중치 (0.5, −1.0, 2.0), 편향 0.2

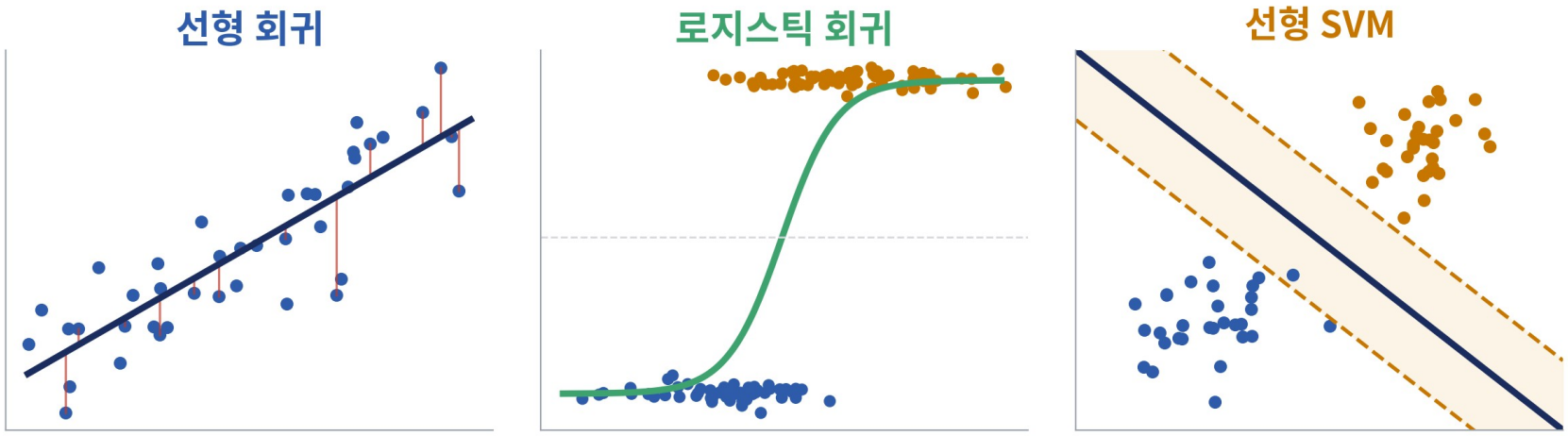
$$a = \sigma\Big(\underbrace{w_1x_1 + w_2x_2 + \cdots + w_nx_n}_{\text{가중 증거}} + \underbrace{b}_{\text{임계값}} \Big) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$



뉴런 하나, 목적함수 셋, 고전 모형 셋

선형 점수 z = wᵀx + b는 같고 목적함수만 바뀐다(분류에서 y = ±1)

$$z = \mathbf{w}^\top \mathbf{x} + b \quad \Longrightarrow \quad \begin{cases} \text{선형 회귀:} & \min \ \frac{1}{N}\sum (y_i - z_i)^2 \\ \text{로지스틱 회귀:} & \min \ \frac{1}{N}\sum \log\left(1 + e^{-y_i z_i}\right) \\ \text{선형 SVM:} & \min \ \frac{1}{N}\sum \max(0,\ 1 - y_i z_i) + \lambda \|\mathbf{w}\|^2 \end{cases}$$



직선 적합, 잔차 제곱

확률 출력, 로그 손실

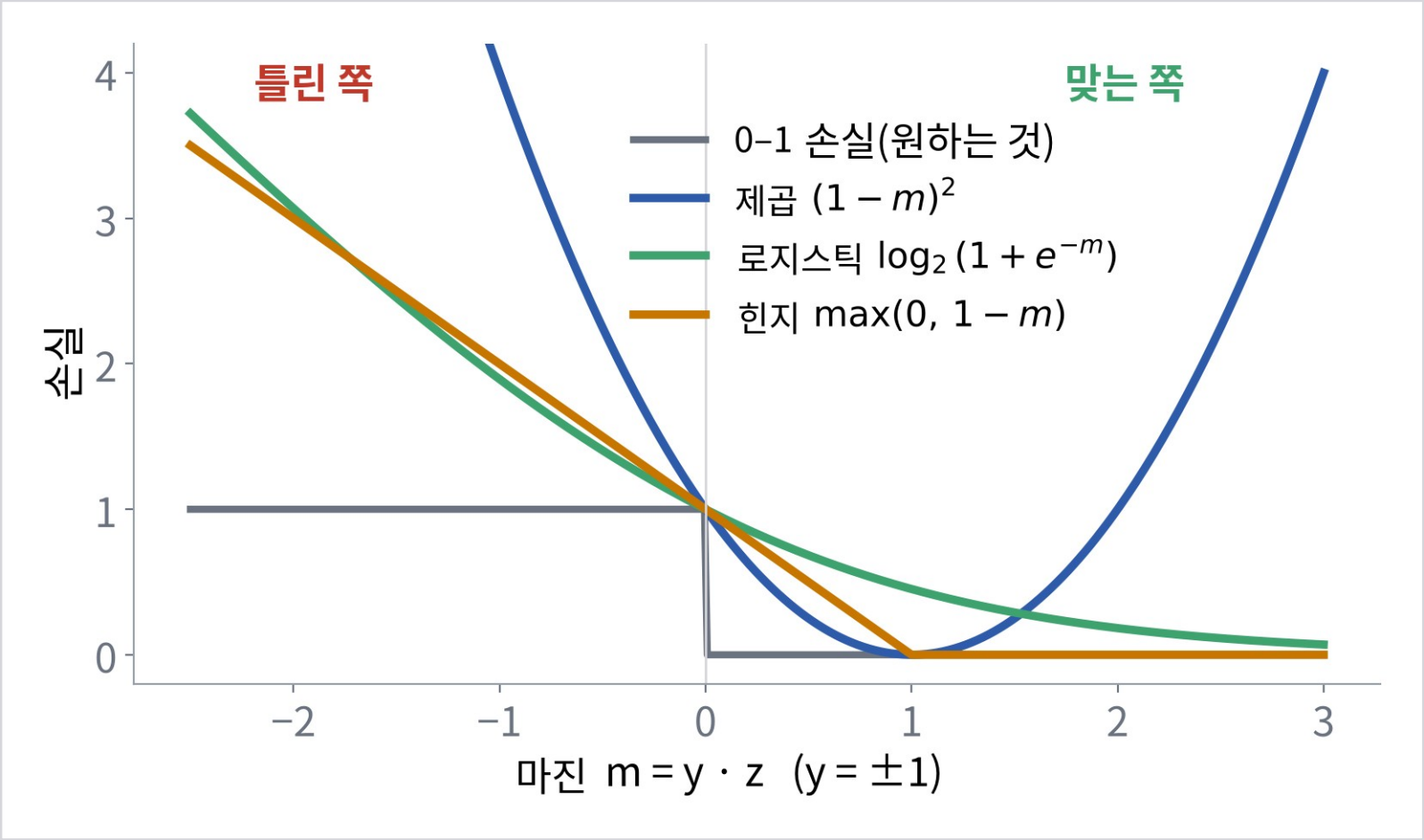
가장 넓은 마진, 힌지 손실

손실마다 마진 오류를 벌하는 방식이 다르다

마진 m = y·z: 양수면 올바른 쪽, 클수록 더 확신

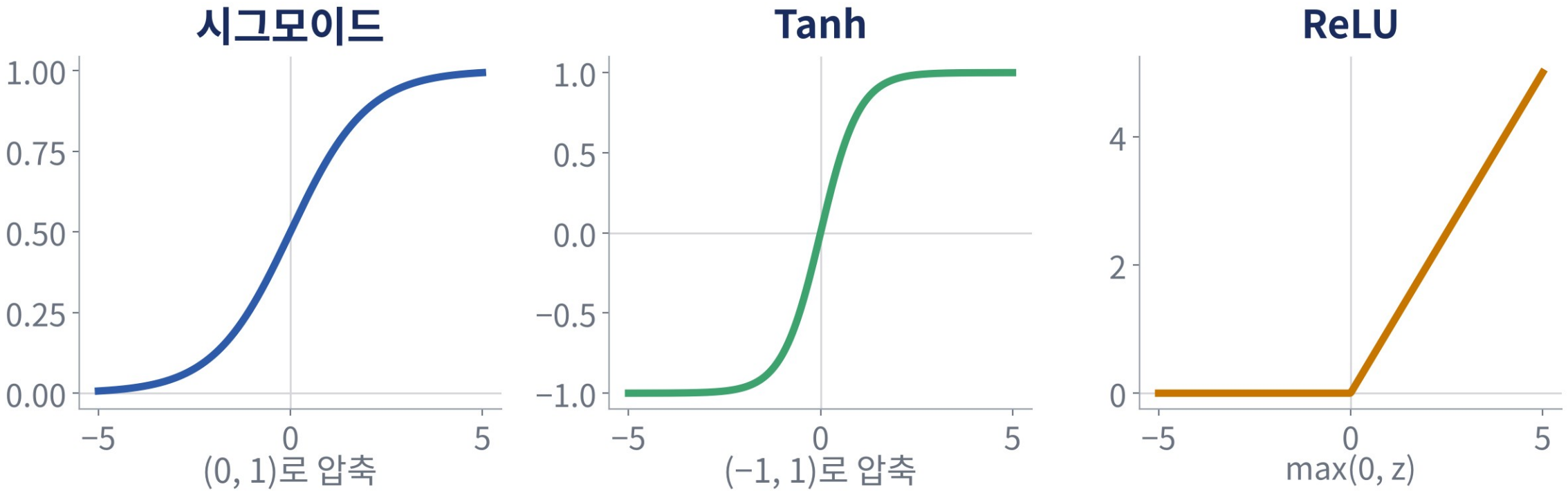
읽는 법

- 모두 0–1 손실을 근사한다
- 제곱 손실: 맞혀도 벌점
- 로지스틱: 완전히 0이 되지 않는다
- 힌지: 마진 1을 넘으면 0
- 신경망 = z까지 학습



활성화 함수는 직선을 구부린다

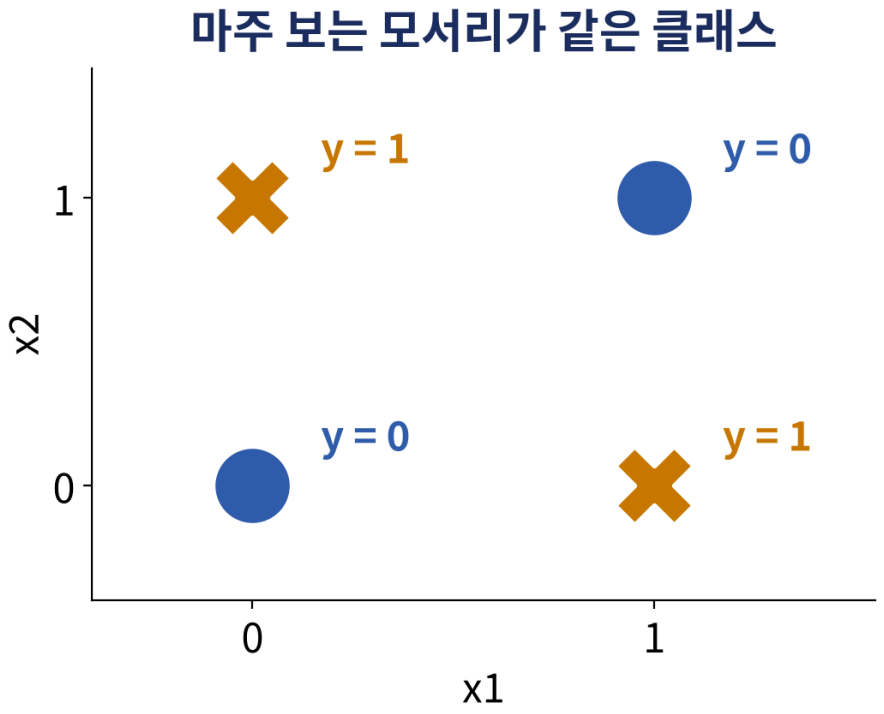
활성화 함수가 없으면 층을 아무리 쌓아도 선형 모형 하나로 무너진다



XOR은 두 입력이 다를 때만 1을 낸다

배타적 논리합(exclusive OR)은 기본 논리 게이트로, 입력 중 정확히 하나만 참일 때 참이다

x1	x2	OR	AND	XOR
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0



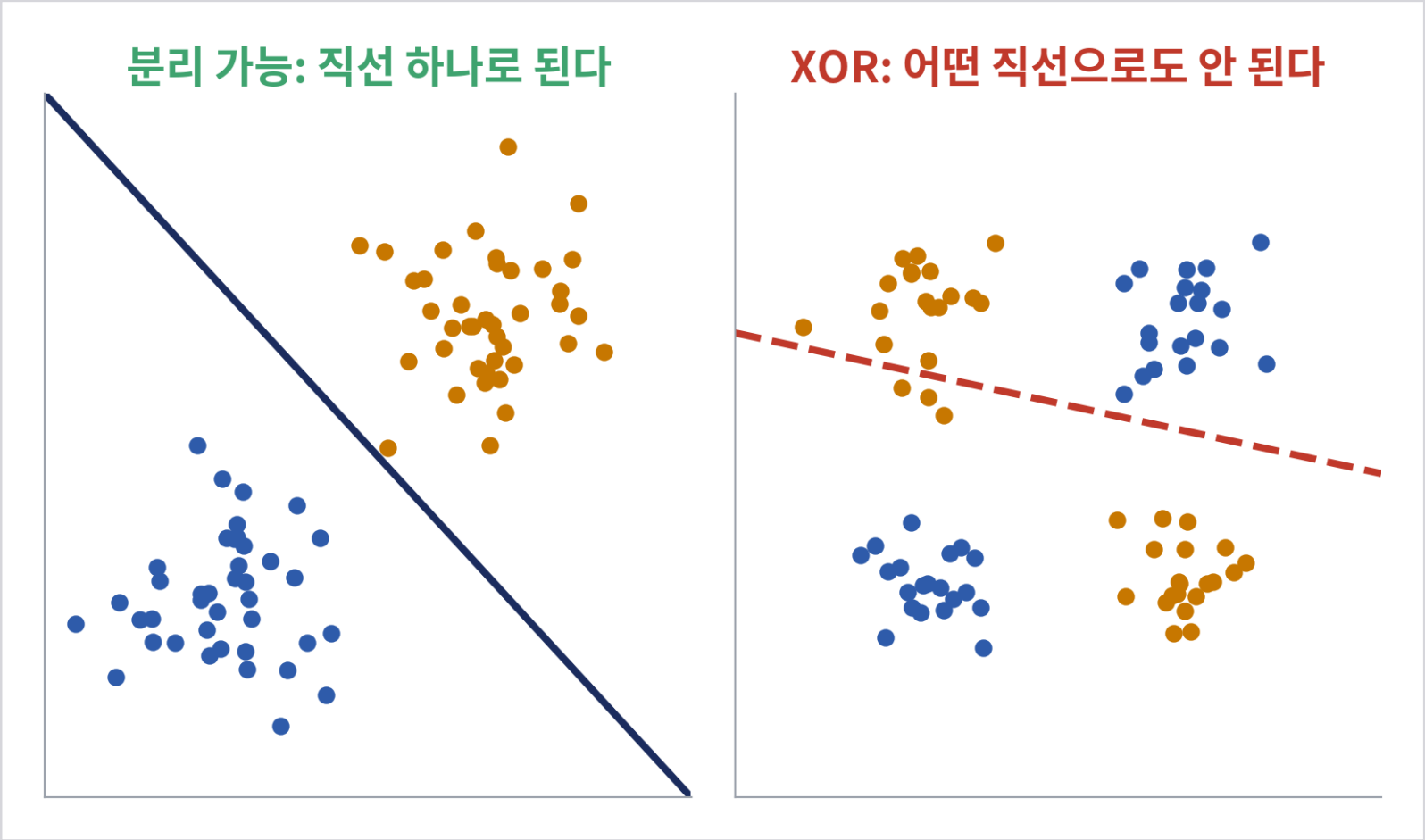
XOR = OR이면서 AND는 아님 · 일상의 예: 스위치가 둘인 복도 전등

뉴런 하나는 직선 경계 하나만 그릴 수 있다

XOR(배타적 논리합): 입력 중 정확히 하나가 양수일 때 양성 클래스

볼 것

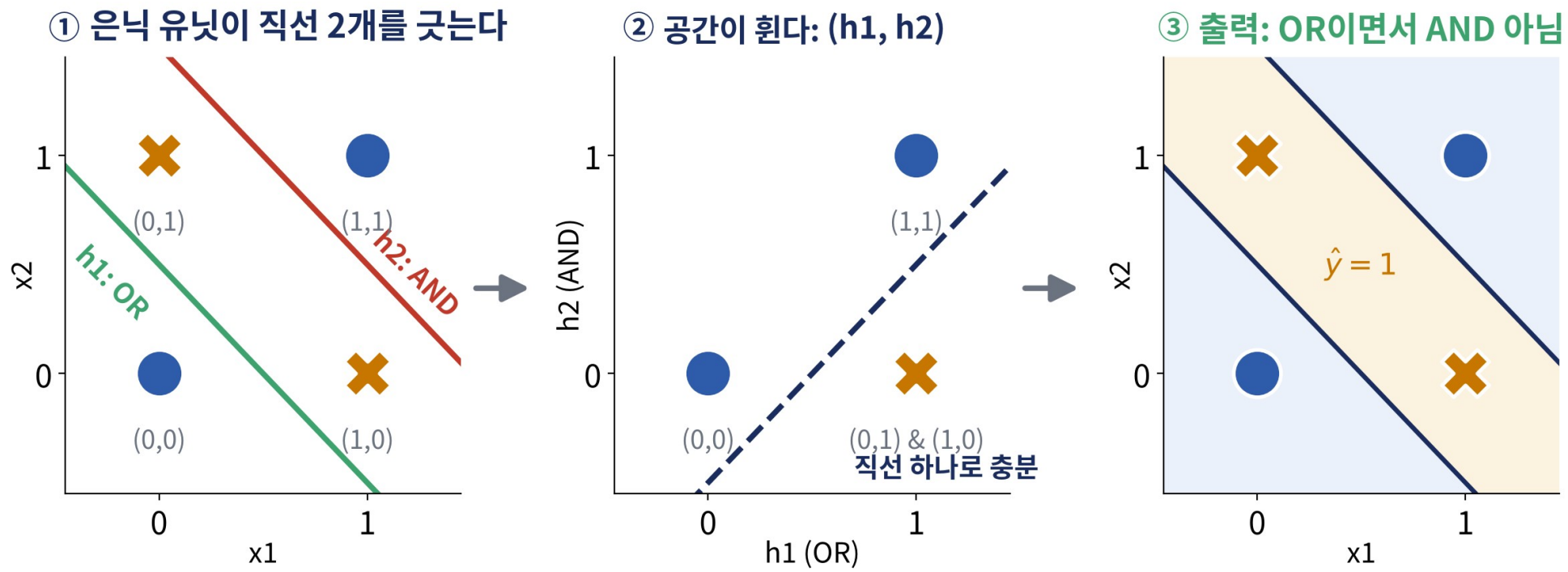
- 왼쪽: 직선 하나로 나뉜다
- 오른쪽: 어떤 직선으로도 안 된다
- 해법: 은닉층을 더한다
- 1969년 Minsky–Papert의 비판



은닉 유닛 둘이 공간을 구부려 직선 하나로 충분하게 만든다

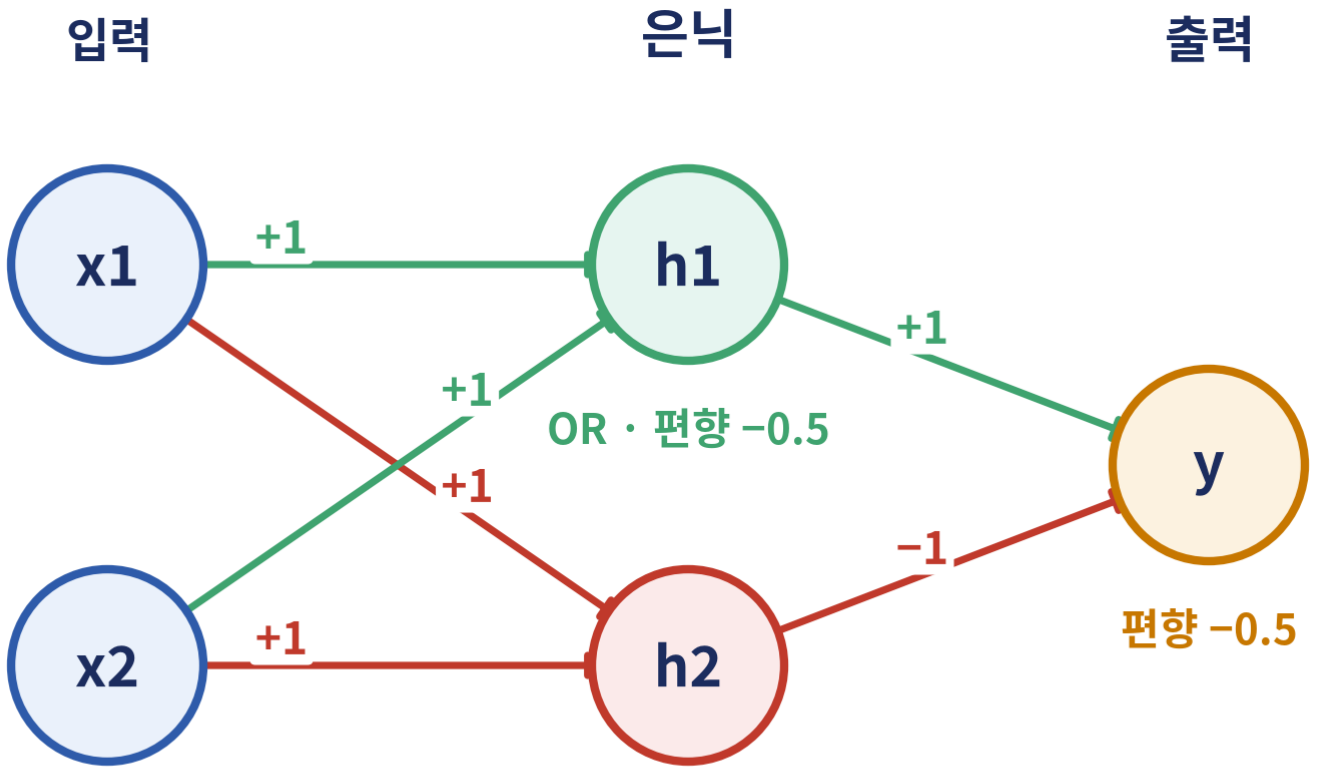
가중치는 손으로 정함; 1[·]은 조건이 맞으면 1, 아니면 0(계단 활성화)

$$h_1 = \mathbf{1}[x_1 + x_2 - 0.5 > 0] \ \ (\mathrm{OR}), \qquad h_2 = \mathbf{1}[x_1 + x_2 - 1.5 > 0] \ \ (\mathrm{AND}), \qquad \hat{y} = \mathbf{1}[h_1 - h_2 - 0.5 > 0]$$



은닉 유닛 둘과 출력 하나로 XOR을 푼다

계단 활성화; 여기서는 가중치와 편향을 손으로 정했지만 학습하면 스스로 찾는다



가중합 + 편향 > 0이면 유닛이 발화(= 1)

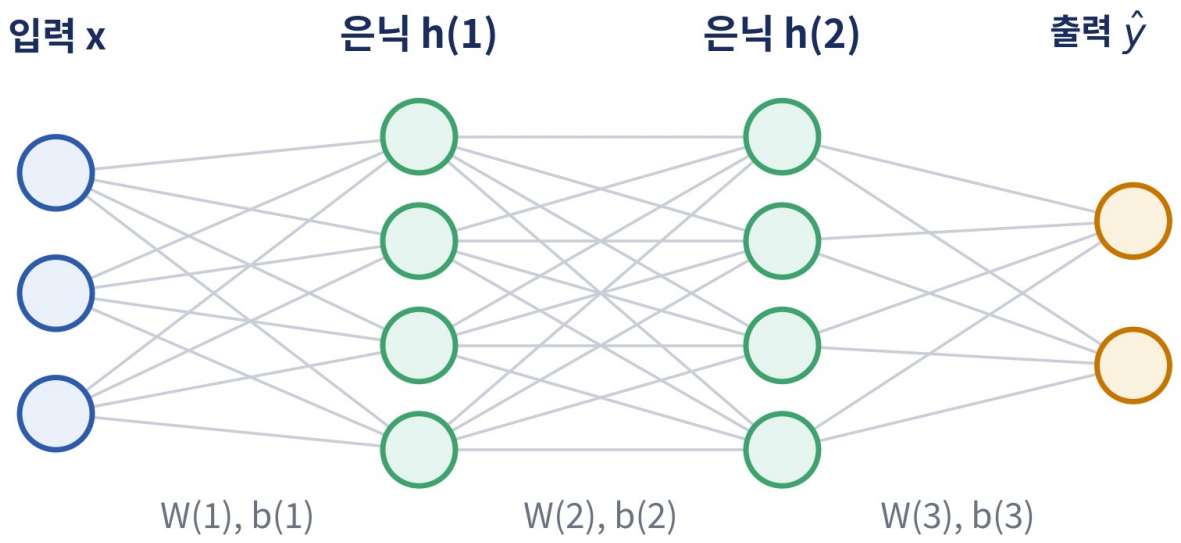
x1 ×2	h1 (OR)	h2 (AND)	ŷ
0 0	0	0	0
0 1	1	0	1
1 0	1	0	1
1 1	1	1	0

$\hat{y} = \text{OR} - \text{AND} = \text{XOR}$

층은 단순한 꺾임을 합성해 복잡한 함수를 만든다

다층 퍼셉트론(MLP, multilayer perceptron): 한 층의 모든 유닛이 다음 층의 모든 유닛에 연결된다

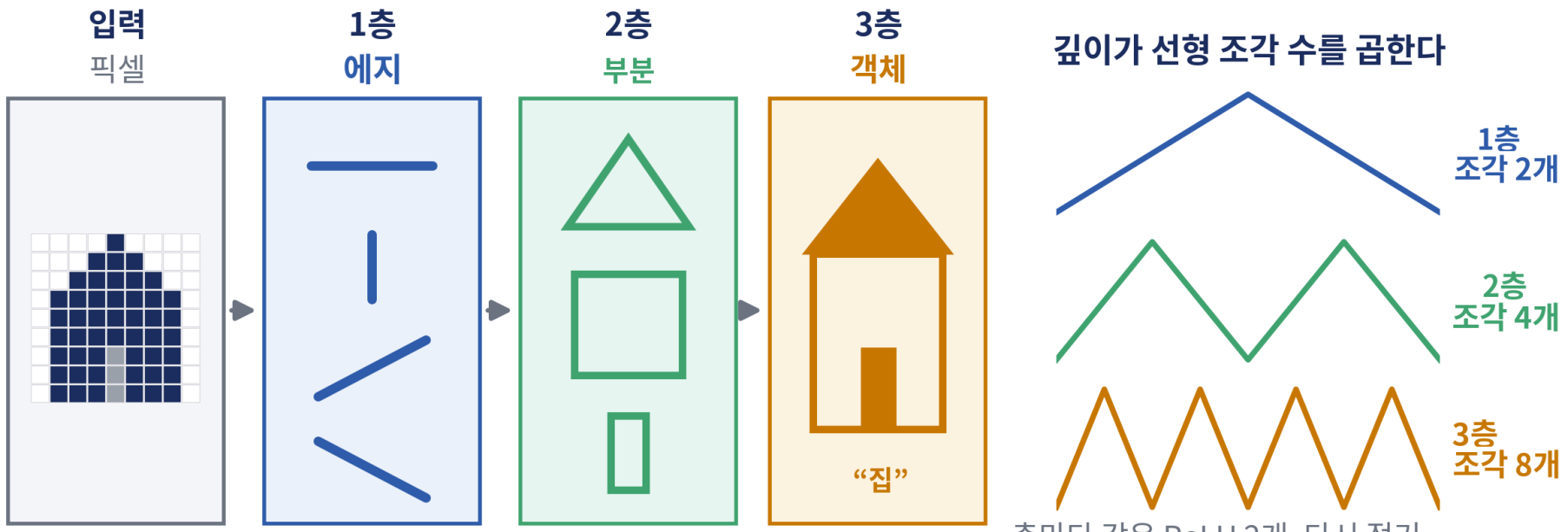
$$\mathbf{h}^{(1)} = \sigma\left(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right), \quad \mathbf{h}^{(2)} = \sigma\left(W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right), \quad \hat{y} = W^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}$$



각 층은 아래층보다 풍부한 특성을 만든다

깊은 신경망은 단순한 함수의 합성이다; 깊이는 크기만이 아니라 재사용을 쌓는다

$$\hat{y} = f(\mathbf{x}) = \underbrace{f^{(3)}}_{\text{객체}}\left(\underbrace{f^{(2)}}_{\text{부분}}\left(\underbrace{f^{(1)}}_{\text{에지}}(\mathbf{x})\right)\right)$$



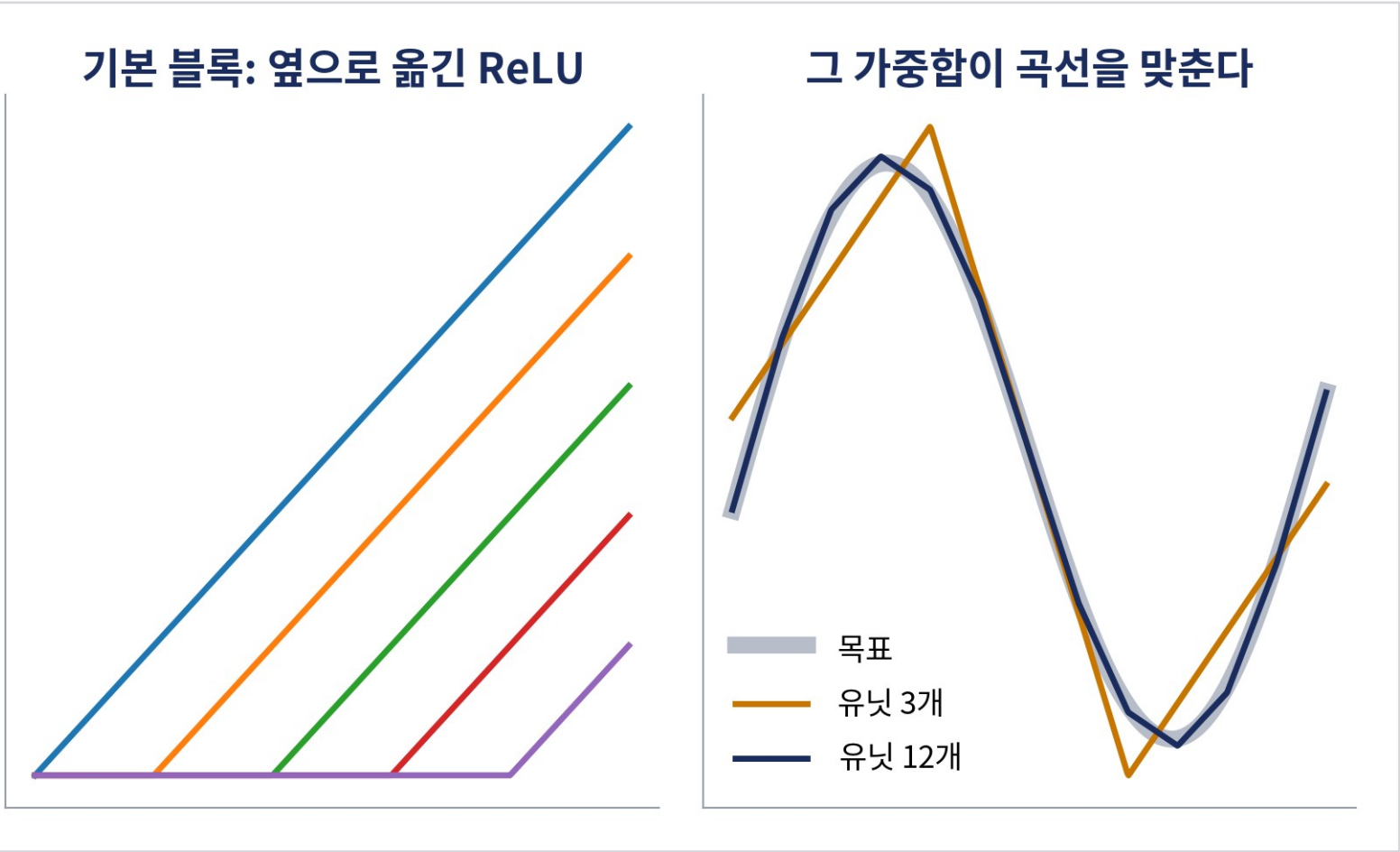
각 층이 앞 층의 특성을 조합한다

은닉 유닛이 충분하면 어떤 곡선이든 근사할 수 있다

보편 근사 정리(universal approximation theorem; Cybenko 1989; Hornik 1991)

직관

- ReLU마다 꺾임 하나를 더한다
- 가중치가 꺾임의 기울기를 정한다
- 유닛이 많을수록 더 정밀하게 맞춘다
- 존재만 말할 뿐 찾는 법은 말하지 않는다



02

2부 · 학습

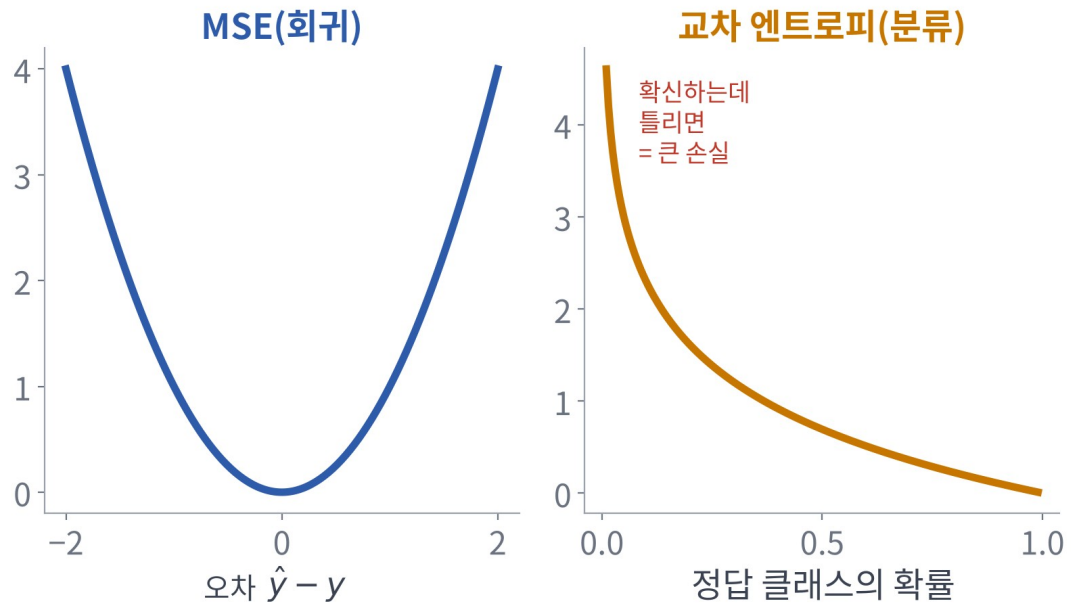
신경망은 어떻게 학습하는가

손실, 경사하강법, 학습률, 역전파

손실 함수는 모형이 얼마나 틀렸는지 점수를 매긴다

회귀는 제곱 오차, 분류는 교차 엔트로피(cross-entropy, 로그 손실)를 쓴다

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 \qquad \mathrm{CE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right]$$



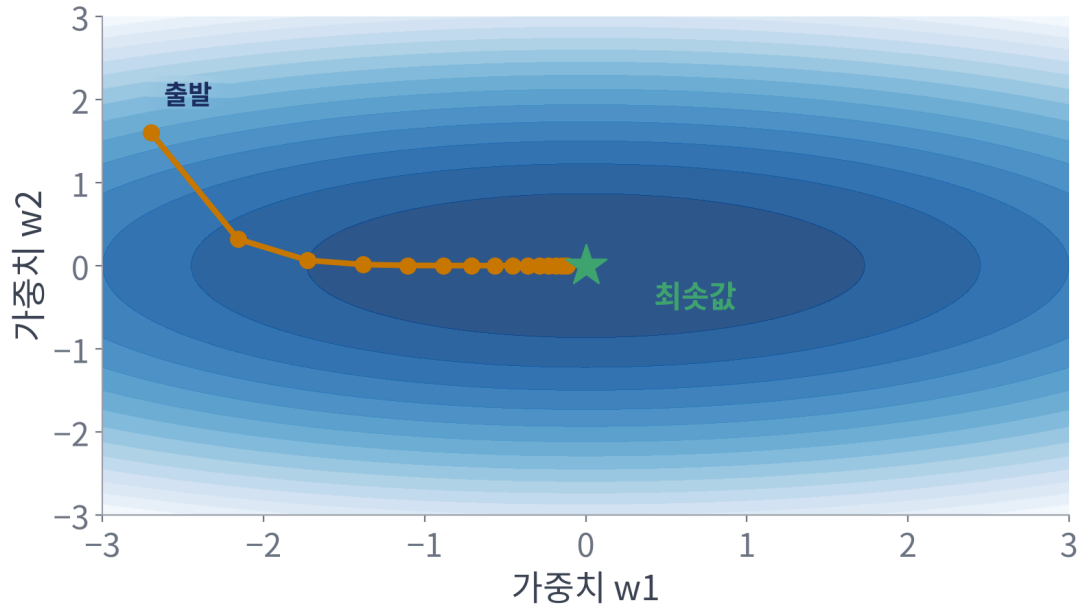
왜 이 둘인가

- MSE는 크게 빗나갈수록 가장 크게 벌한다
- CE는 모형이 확신하면서 틀리면 폭발한다
- 학습 = 이 숫자를 작게 만드는 것

경사하강법은 기울기 반대 방향으로 거듭 한 걸음씩 간다

θ(세타) = 모든 가중치와 편향; η(에타) = 학습률

$$\theta \leftarrow \theta - \underbrace{\eta}_{\text{보폭}} \underbrace{\nabla_\theta L(\theta)}_{\text{오르막 방향}}$$

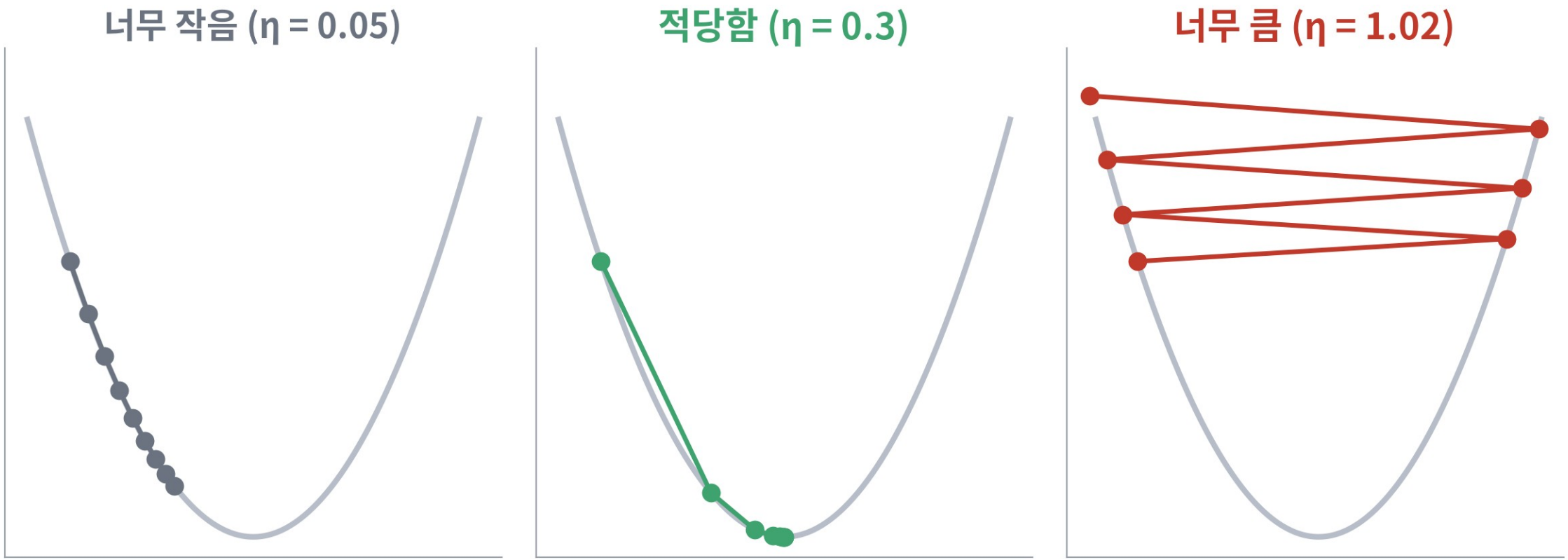


안개 속 등산객

- 발밑의 기울기를 느낀다
- 내리막으로 한 걸음, 다시 느낀다
- 미니배치 SGD: 작은 무작위 표본으로 기울기를 잰다

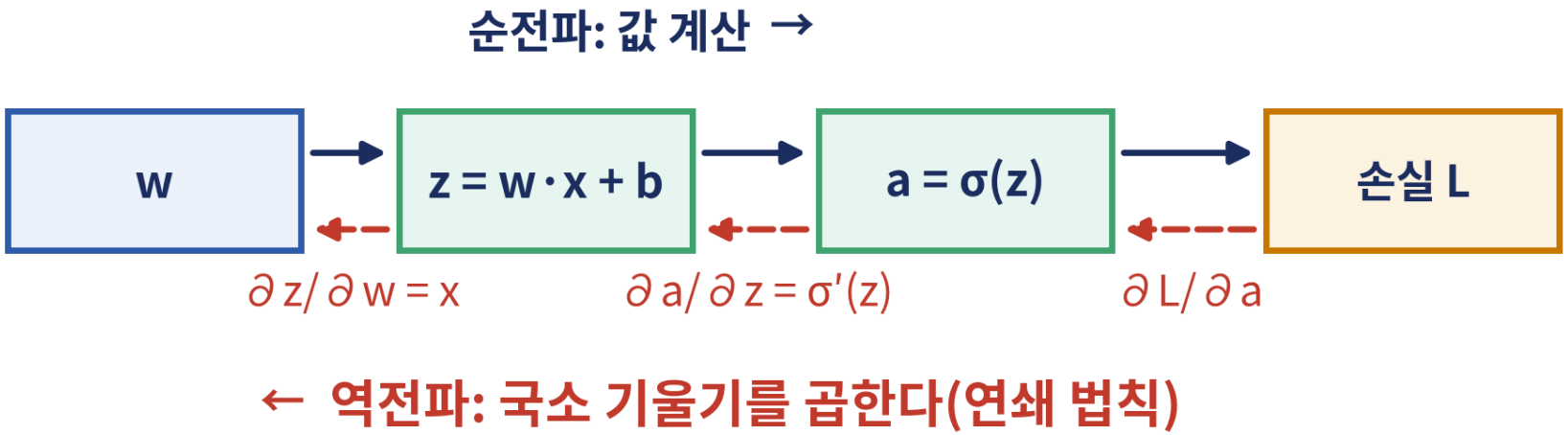
학습률이 학습의 수렴 여부를 정한다

같은 손실 $L(w) = w^2$, 같은 출발점, 세 가지 보폭



역전파는 연쇄 법칙으로 책임을 거꾸로 보낸다

순전파 한 번으로 값을, 역전파 한 번으로 모든 기울기를 계산한다



$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

손실이 더 내려가지 않을 때까지 네 단계를 반복한다

전체 데이터를 한 번 훑으면 에폭(epoch), 미니배치 한 번은 반복(iteration)이다

1. 순전파

예측

미니배치를 신경망에 통과시켜 ŷ를 얻는다

x → ŷ

2. 손실

측정

ŷ를 실제 레이블 y와 비교한다

L(ŷ, y)

3. 역전파

책임 배분

역전파가 모든 가중치의 ∂L/∂θ를 계산한다

연쇄 법칙

4. 갱신

한 걸음

옵티마이저가 가중치를 내리막으로 조금 옮긴다

θ ← θ − η∇L

프레임워크(PyTorch, JAX)가 3–4단계를 자동화하고, 1–2단계는 사람이 설계한다

03

3부 · 실전

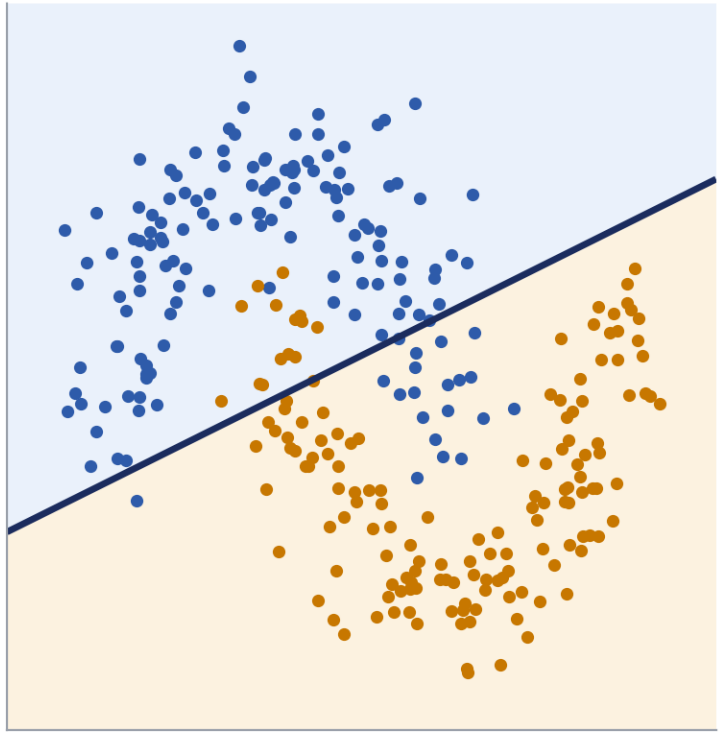
신경망이 제대로 작동하게 하기

용량, 과적합, 합리적인 기본값

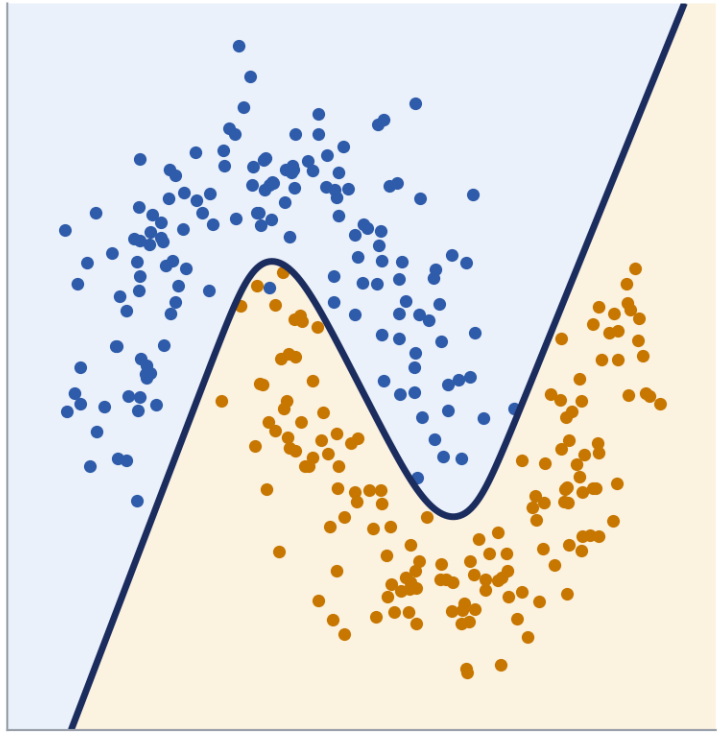
은닉 유닛이 많을수록 결정 경계가 휘어질 수 있다

반달 두 개(two-moons) 데이터, 은닉층 하나(tanh), NumPy로 경사하강법 학습

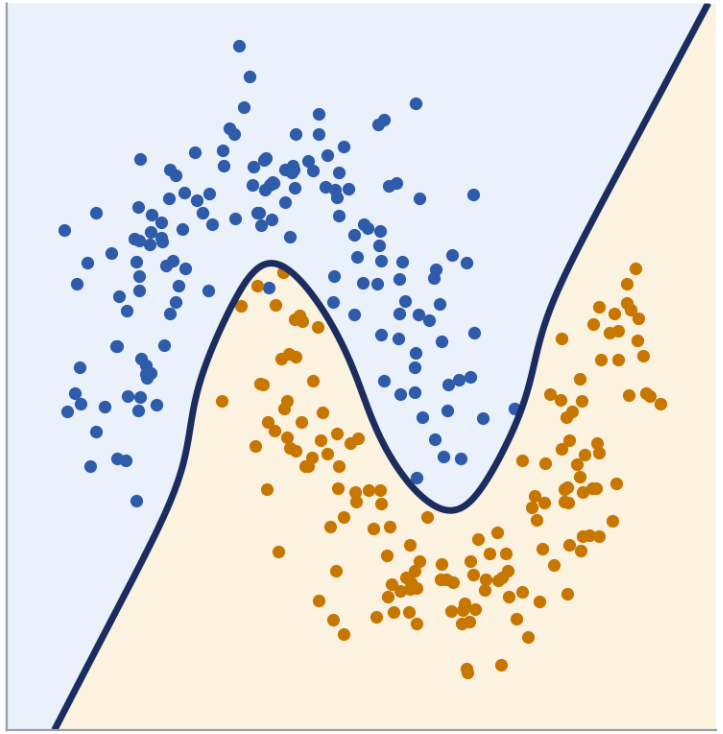
은닉 유닛 1개 · 정확도 89%



은닉 유닛 3개 · 정확도 100%



은닉 유닛 16개 · 정확도 100%

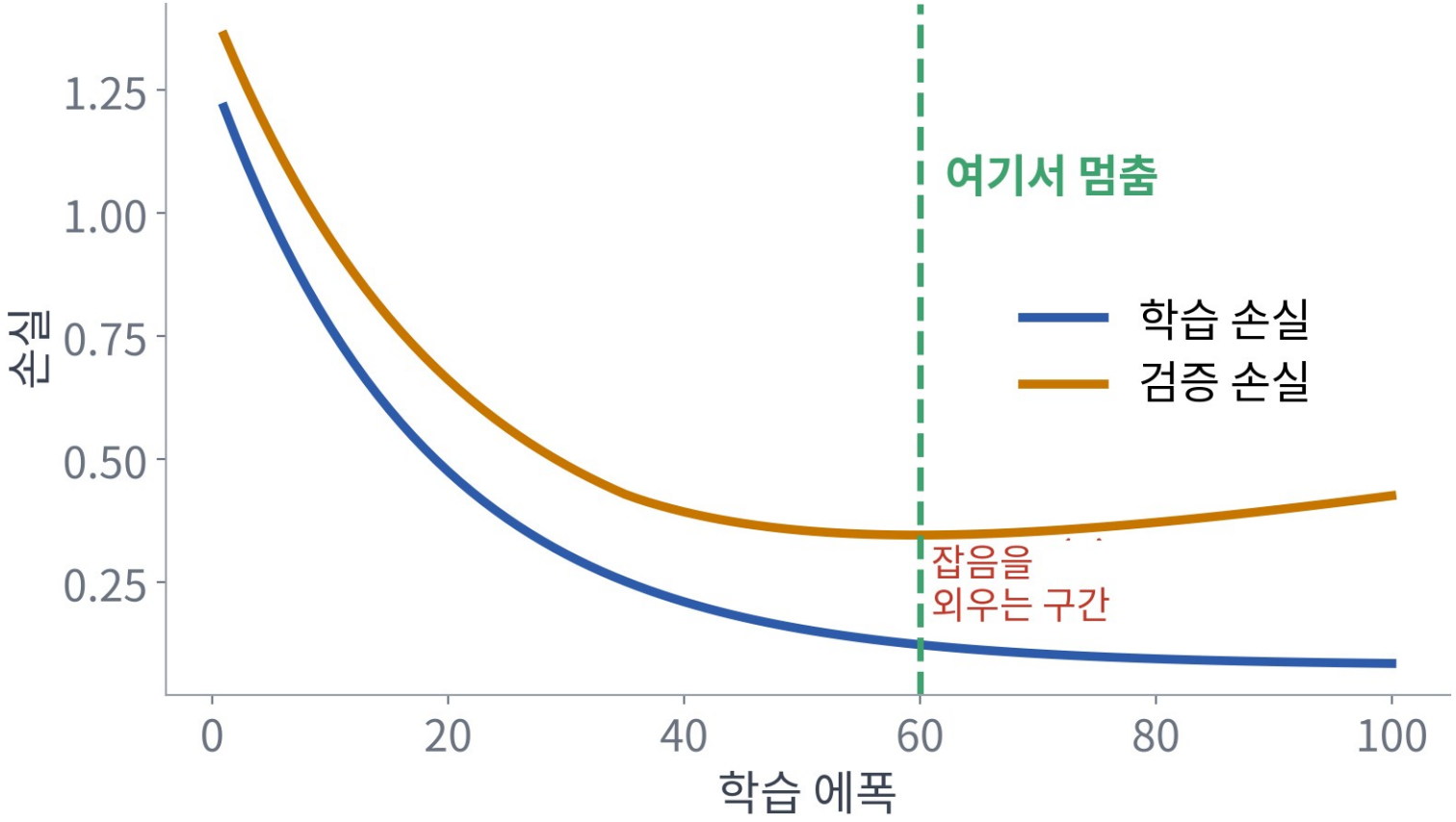


검증 손실을 보면 암기가 시작되는 때를 알 수 있다

학습 손실은 늘 내려가지만, 잡음까지 맞추기 시작하면 검증 손실이 다시 오른다

처방

- 조기 종료(early stopping)
- 데이터 추가 또는 증강
- 가중치 감쇠(weight decay, L2)
- 드롭아웃(dropout)



합리적인 기본값이면 대부분의 신경망이 학습된다

규칙이 아니라 출발점이다 — 검증 세트로 조정한다

선택 항목	기본값	이유
은닉층 활성화	ReLU (또는 GELU)	계산이 싸고, $z > 0$에서 기울기가 사라지지 않는다
출력 + 손실	선형 + MSE / softmax + CE	목표 변수의 형태에 맞춘다
옵티마이저	Adam, $\eta \approx 1e\text{-}3$	가중치마다 보폭을 조절한다
초기화	He / Xavier	층을 거쳐도 신호 크기를 안정적으로 유지한다
정규화	가중치 감쇠 + 조기 종료	과적합을 적은 비용으로 통제한다

04

4부 · 예시

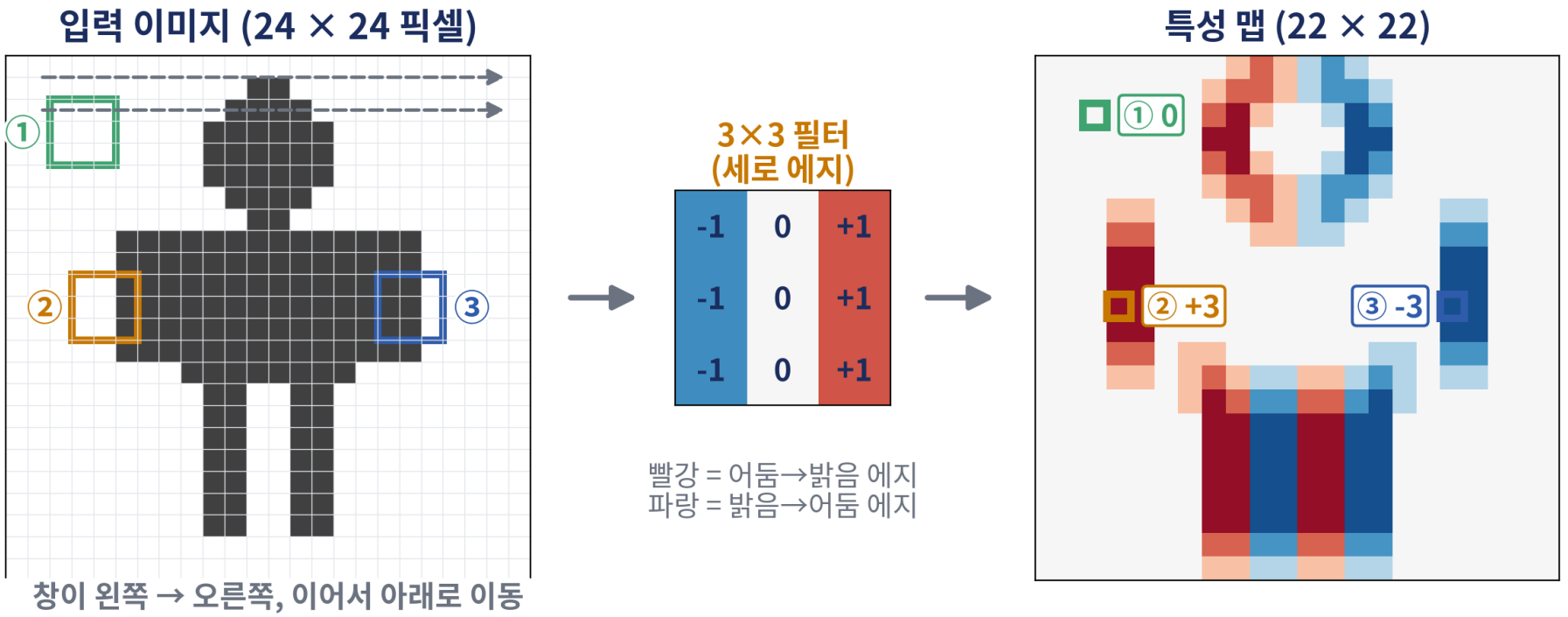
현대 신경망 구조와 금융

구성 요소는 같고 연결 방식만 다르다: CNN, RNN, 오토인코더, 트랜스포머

미끄러지는 필터가 이미지의 국소 패턴을 찾는다

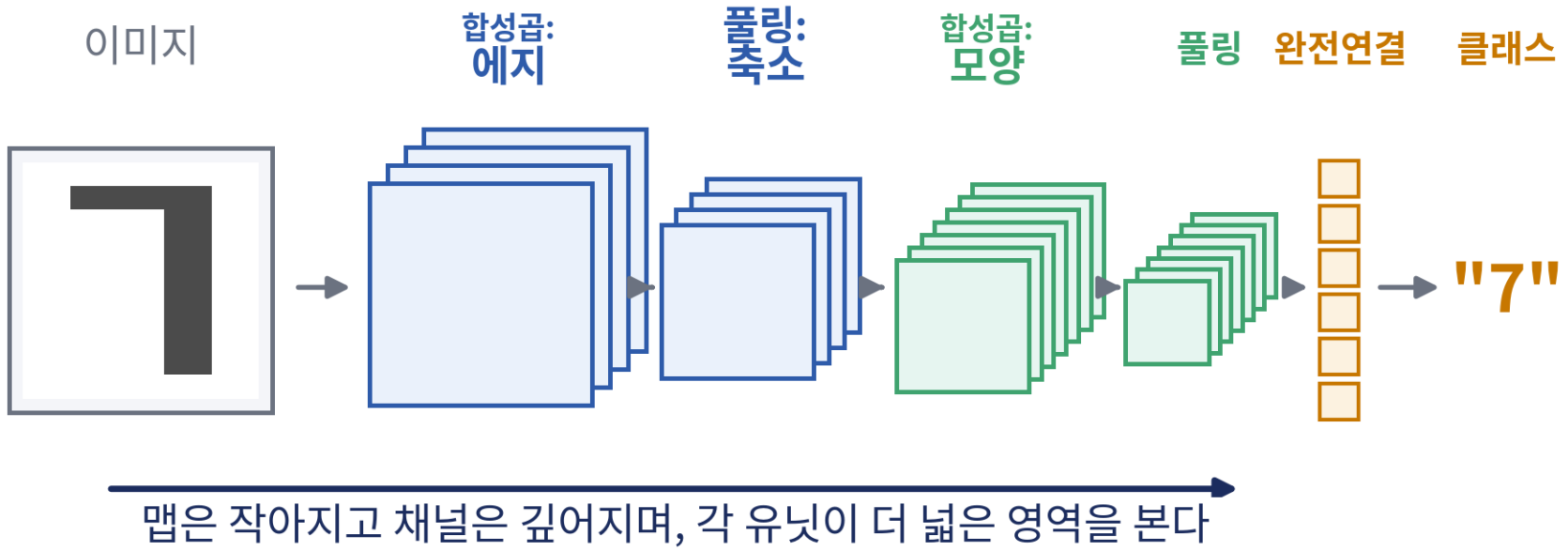
합성곱(convolution): 같은 3×3 가중치를 모든 위치에서 재사용한다(가중치 공유)

$$\text{특성}(i,j) = \sum_{m=0}^{2}\sum_{n=0}^{2} \text{이미지}(i+m,\, j+n) \cdot K(m,n)$$



합성곱을 쌓으면 픽셀이 개념이 된다

풀링은 각 맵을 절반으로 줄이고, 깊은 층일수록 더 넓은 영역을 본다(수용 영역)



금융: Jiang, Kelly & Xiu (2023) — 가격 차트 이미지에 CNN을 적용해 수익률을 예측

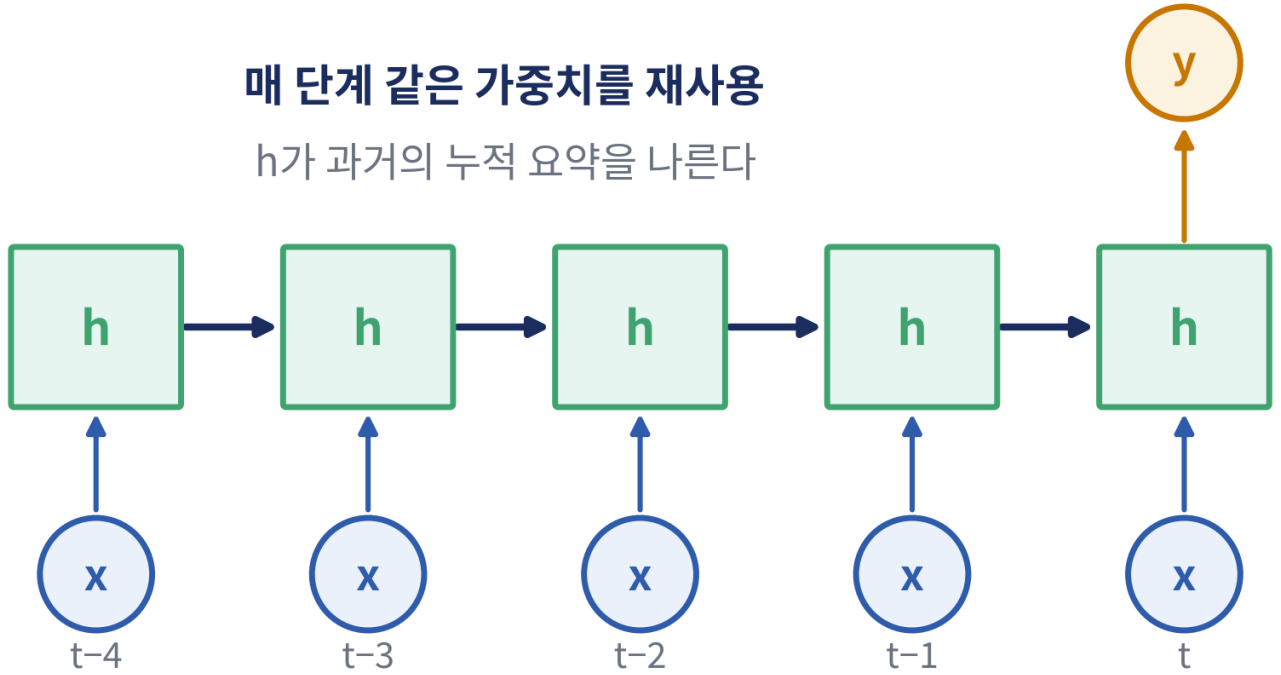
순환 신경망은 단계에서 단계로 기억을 넘긴다

시계열과 시퀀스: 관측 순서가 중요하다

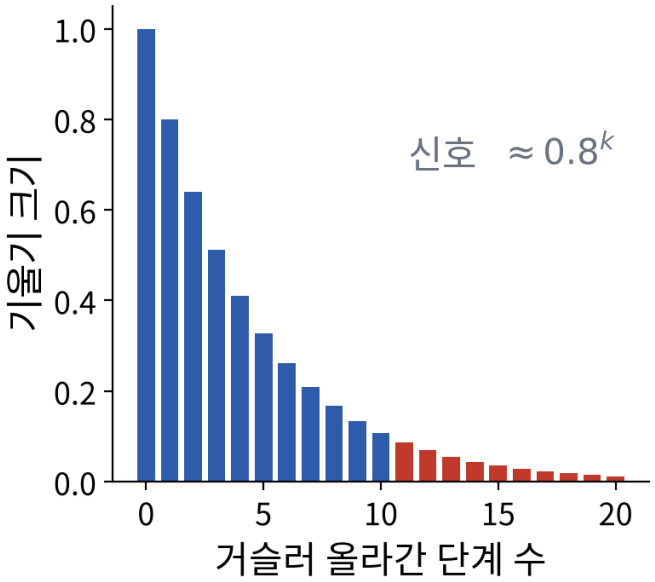
$$h_t = \tanh\left(W_h\, h_{t-1} + W_x\, x_t + b\right), \qquad \hat{y}_t = W_y\, h_t$$

매 단계 같은 가중치를 재사용

h가 과거의 누적 요약을 나른다



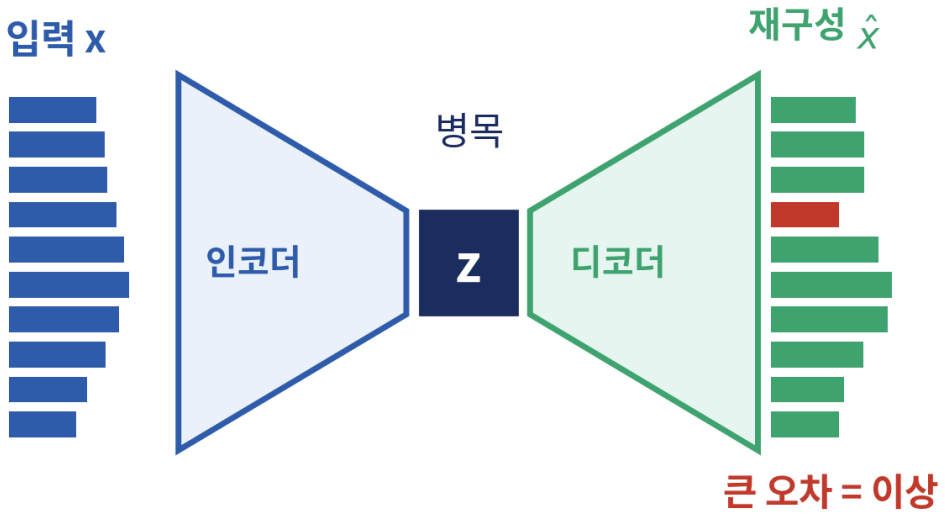
단순 RNN은 잊는다



오토인코더는 병목을 거쳐 데이터를 압축한다

목표가 입력 자체이므로 레이블이 필요 없다(비지도 학습)

$$\mathbf{z} = f_{\text{enc}}(\mathbf{x}), \qquad \hat{\mathbf{x}} = f_{\text{dec}}(\mathbf{z}), \qquad \min \|\mathbf{x} - \hat{\mathbf{x}}\|^2 \quad (\dim \mathbf{z} \ll \dim \mathbf{x})$$



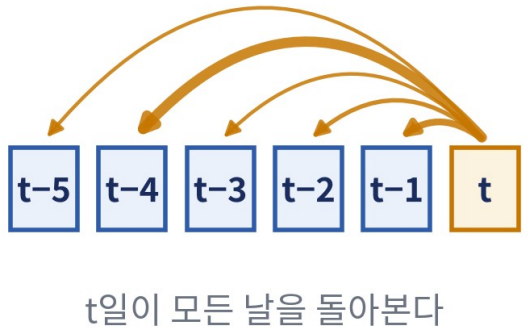
금융 활용

- 수익률 곡선: z ≈ 수준, 기울기, 곡률 (비선형 PCA)
- 사기 탐지·자금세탁방지(AML): 재구성 오차가 크면 이상 거래로 표시

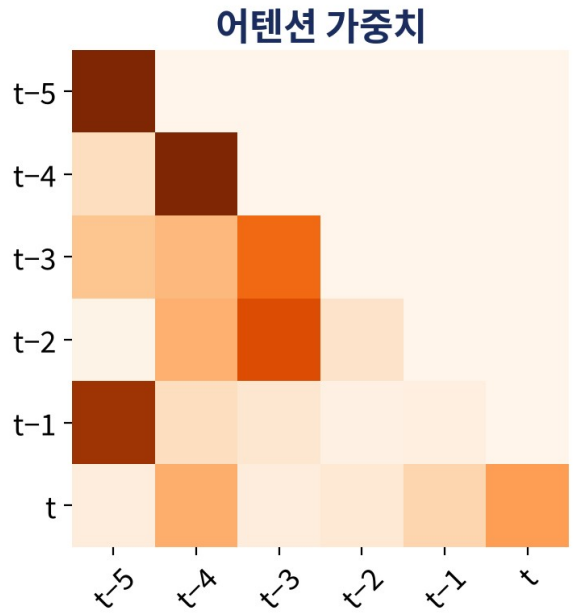
셀프 어텐션은 각 단계가 다른 모든 단계에 가중치를 매기게 한다

셀프 어텐션(self-attention): 각 토큰이 다른 모든 토큰과의 관련도를 매긴 뒤 가중평균을 취한다

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \qquad Q = XW_Q,\ K = XW_K,\ V = XW_V$$



t일이 모든 날을 돌아본다



금융 활용

- 텍스트: 공시와 뉴스에 FinBERT 방식 감성 분석
- 시퀀스: 최근 며칠만이 아니라 관련 있는 과거 날짜에 주목한다

원조 트랜스포머는 인코더와 디코더를 짝지운다

Vaswani et al. (2017), Attention Is All You Need의 그림 1

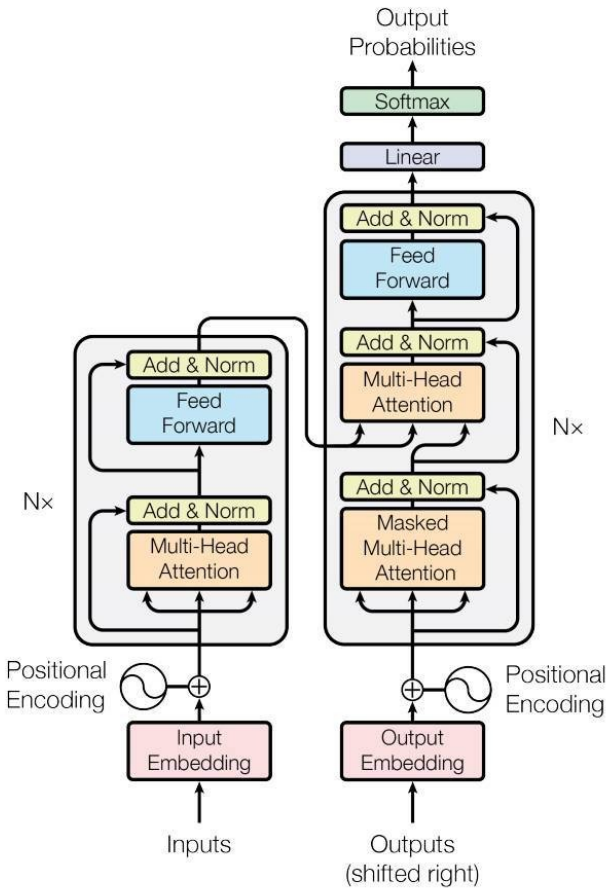


그림 읽는 법

- 왼쪽 탑 = 인코더: 입력 전체에 셀프 어텐션
- 오른쪽 탑 = 디코더: 마스크드 어텐션은 앞선 출력만 본다
- 가운데 어텐션 블록은 인코더에 질의한다(교차 어텐션)
- Add & Norm = 잔차 연결 + 층 정규화
- 위치 인코딩이 단어 순서를 넣는다; N = 6층
- BERT는 인코더를, GPT는 디코더를 남긴다

금융 과제마다 맞는 신경망이 있다

발표된 사례; 구조 이름은 이 강의의 1–4부를 가리킨다

문제	구조	신경망이 하는 일
수익률 예측	MLP	Gu, Kelly & Xiu (2020): 기업 특성 변수에서 선형 모형을 앞선다
팩터 모형	오토인코더	Gu, Kelly & Xiu (2021): 잠재 팩터, 특성 변수로 베타 추정
사기 탐지 / AML	오토인코더	잘 재구성하지 못하는 거래를 표시한다
텍스트 신호	트랜스포머	FinBERT (Araci 2019): 금융 텍스트에서 감성 추출
헤지	MLP 정책	딥 헤징(Buehler et al. 2019): 거래비용 아래 헤지 전략을 학습

요약

신경망은 내리막을 따라 맞춘 유연한 함수다

1

뉴런 = 가중합 + 활성화 함수

비선형성이 있어야 깊이가 의미를 갖는다.

2

층 = 꺾임의 합성

너비와 깊이가 모형이 그릴 수 있는 경계의 복잡도를 정한다.

3

학습 = 손실 + 역전파 + 경사하강법

검증 손실을 보고 멈출 때를 정한다.

모든 구조 = 뉴런 + 목적함수 + 경사하강법